

Lecture 02

Data Representation

Comp_Sci 213

Introduction to Computer Systems

Kate Smith – Fall 2026

Slides adapted from:

Ghena, St-Amour, Hardavellas, Bustamente (Northwestern), Bryant, O'Hallaron (CMU), Garcia, Weaver (UC Berkeley)

Announcement!

Tech Career Fair

Wednesday, September 30, 2026

12:00 PM – 4:00 PM

The Tech Career Fair connects employers with Northwestern undergraduate, master's, PhD students, and alumni interested in careers across technology, engineering, and design. Organizations recruiting in areas such as artificial intelligence, machine learning, software engineering, data science, cybersecurity, robotics, UX/UI design, embedded systems, semiconductors, and related fields are encouraged to attend.

Please remember:

- Pre-registration through Handshake is encouraged for all career fairs, though students may register on-site the day of the event.
- A valid Northwestern Wildcard is required for entry.
- Professional attire is recommended. Students are encouraged to dress professionally and present themselves in a manner appropriate for networking with employers.
- Research employers before attending to make the most of your conversations.
- Search [McCormickConnect](#) and [Handshake](#) for available positions and apply for the position(s) ahead of time so companies know you're interested.
- Bring copies of your resume and business cards! Your resume should be uploaded into McCormickConnect.
- Check Handshake regularly, as participating employers and opportunities may be updated leading up to each event.
- Career fair attendance is open to Northwestern undergraduate students, graduate students, and alumni unless otherwise noted.

Announcement 2

- PMs will reach out to you
- Class schedule has information on office hours
- Piazza has information about connecting to Moore / completing labs
 - Lab 1 due Oct 13
- HW 1 will be released tomorrow!
 - Integer and floating point representations
 - Due Oct 15

- **Running a program**
- Representing numbers with binary

Hello World

- What happens when you run “hello” on your system?
 - And *why* does it happen?

```
/*  
 * hello world  
 */  
#include <stdio.h>  
  
int main()  
{  
    printf("hello, world\n");  
}
```

- **Goal:** introduce key concepts, terminology, and components

Compiling **hello**

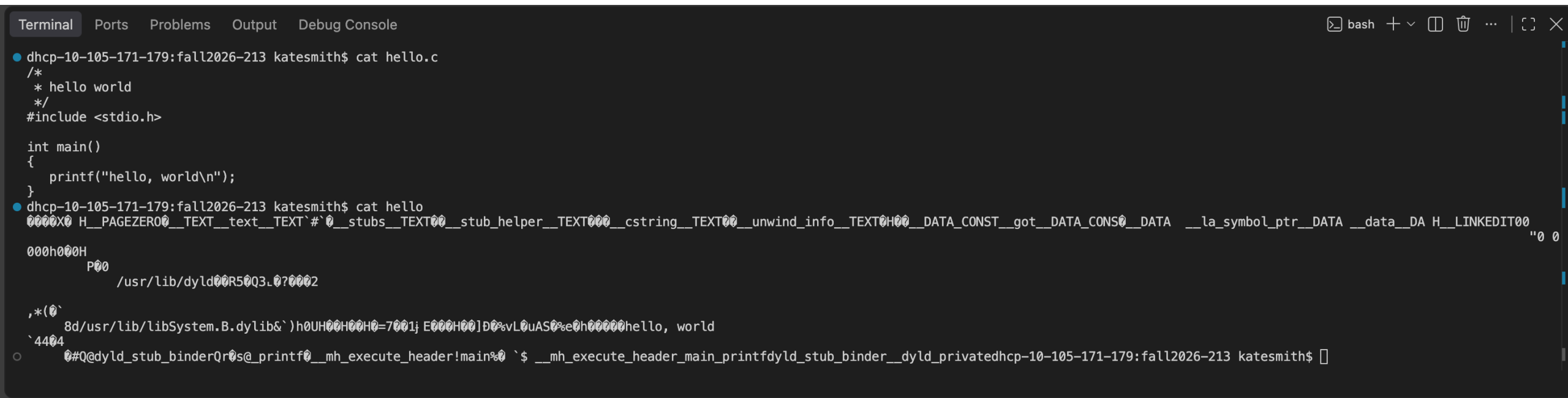
- Compiling **hello**

```
unix> gcc -o hello hello.c
```

- GCC is our compiler
 1. It takes our source code (**hello.c**)
 - A text file containing characters
 - Text file = readable by humans
 2. And translates (or compiles) it into **assembly code**
 - A text representation of x86 instructions that a processor can run
 - Here, not explicitly stored in a file (there's another flag that can do that)
 - We'll be working with assembly a lot this quarter
 3. Then translates (assembles) that into an executable (**hello**)
 - A binary file containing x86 **machine code** that the processor understands
 - Binary file = not meant to be read by humans
 - Our file is named **hello** because of the **-o flab**

Examining the files

Quite a difference between hello.c and hello!



```
Terminal  Ports  Problems  Output  Debug Console
dhcp-10-105-171-179:fall2026-213 katesmith$ cat hello.c
/*
 * hello world
 */
#include <stdio.h>

int main()
{
    printf("hello, world\n");
}
dhcp-10-105-171-179:fall2026-213 katesmith$ cat hello
00000000 H__PAGEZERO__TEXT__text__TEXT`#`__stubs__TEXT__stub_helper__TEXT__cstring__TEXT__unwind_info__TEXT__DATA_CONST__got__DATA_CONST__DATA __la_symbol_ptr__DATA __data__DATA __bss__DATA __LINKED__
00000000 H
00000000 P
00000000 /usr/lib/dyld00R50Q3L0?0002

,*(0`
8d/usr/lib/libSystem.B.dylib&`)h0UH00H00H0=7001j E000H00]D0%vL0uAS0%e0h000000hello, world
`4404
0#0@dyld_stub_binder0r0s@_printf0__mh_execute_header!main%0`$ __mh_execute_header_main_printfdyld_stub_binder__dyld_privatedhcp-10-105-171-179:fall2026-213 katesmith$
```

***Note: this example was run on a Macbook pro that uses a 2.4 GHz 8-Core Intel Core i9 processor -> x86*

Running **hello**

- Running **hello**

```
unix> ./hello  
hello, world  
unix>
```

- What does the shell do?
 - Prints a prompt
 - Waits for you to type a command
 - Interpret the command
 - Then loads and runs the **hello** program
- What happens at the hardware level?

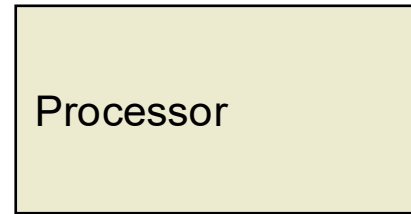
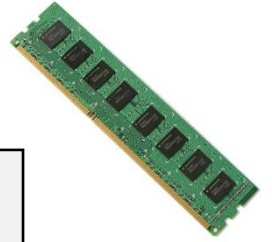
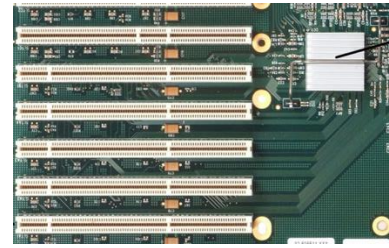
Hardware organization



Processor: Executes instructions stored in main memory

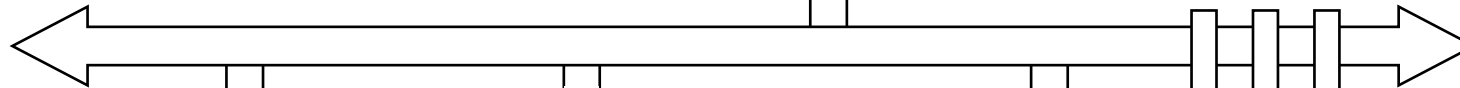
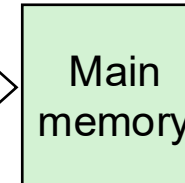
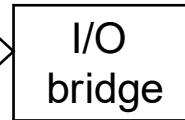
Buses: wires that transfer data

Main mem.: Temporary storage device. Holds both a program and the data it manipulates.



System bus

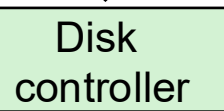
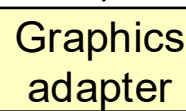
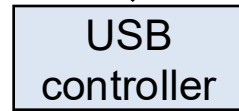
Memory bus



I/O bus



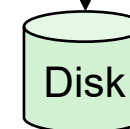
Ethernet HDMI USB-A USB-C



Expansion slots for other devices such as network adapters

Keyboard Mouse

Display



hello executable stored on disk

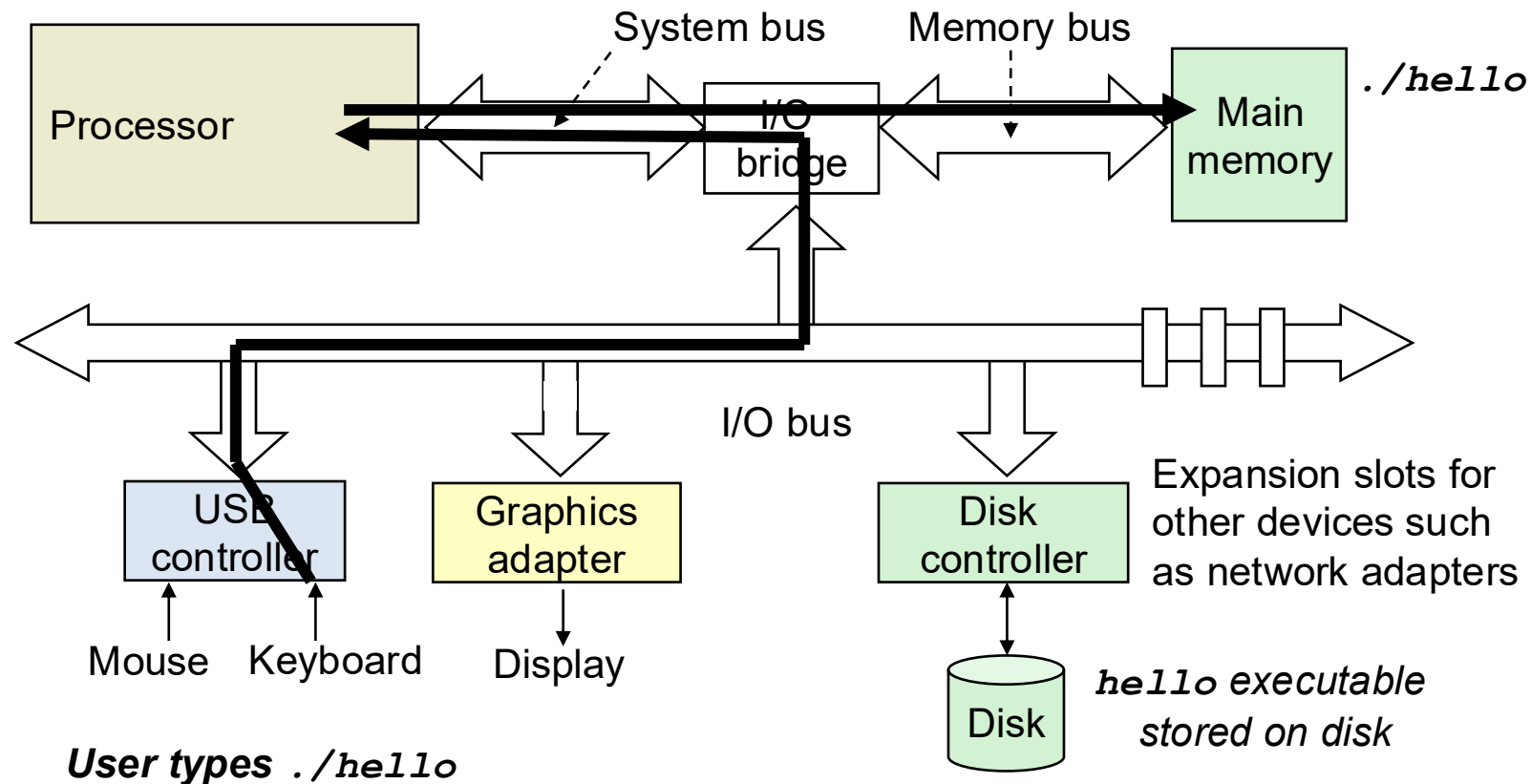


Input/Output (I/O) Devices:
System connections to outside world.

Disk: Persistent storage device

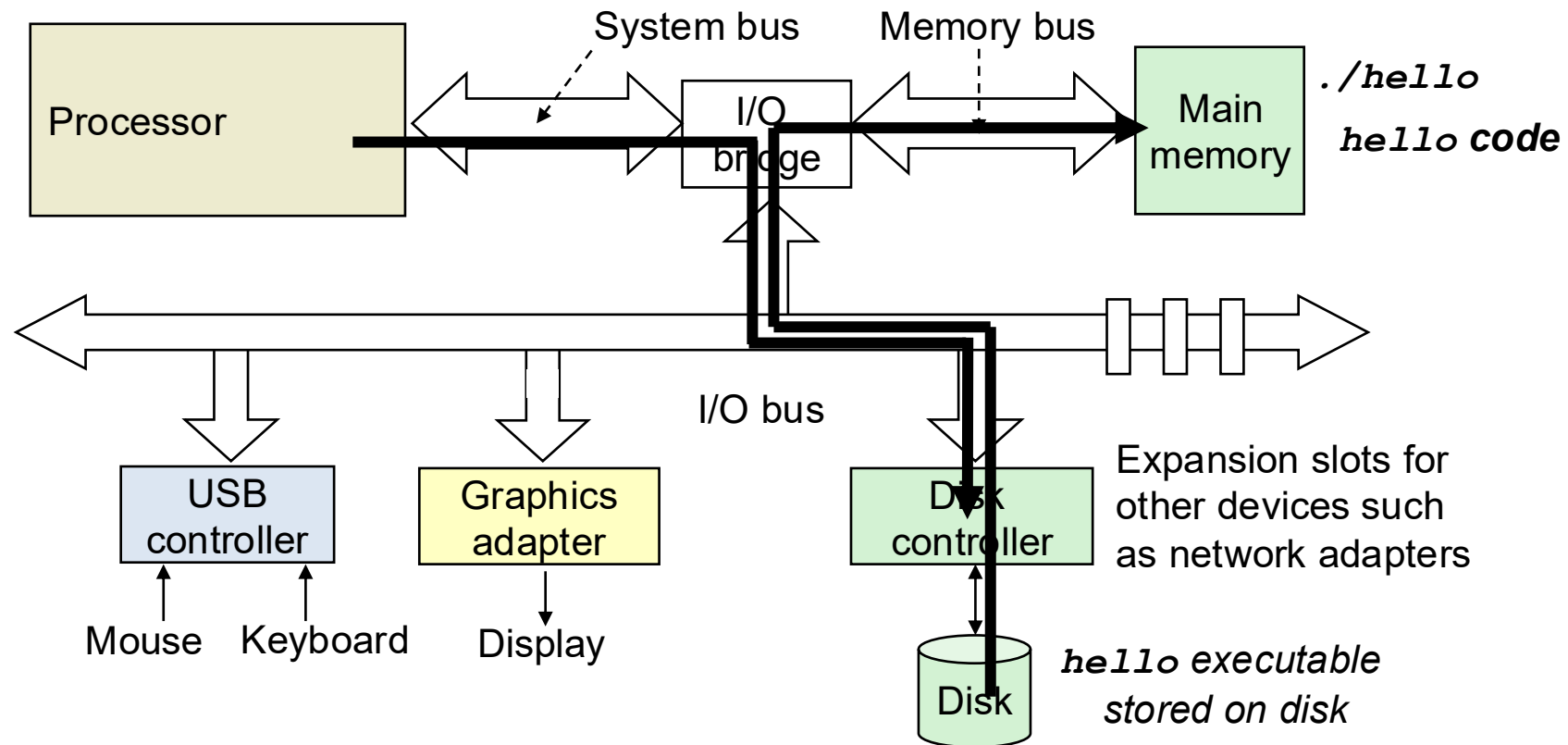
Running `hello`

Reading the `./hello` command
from the keyboard



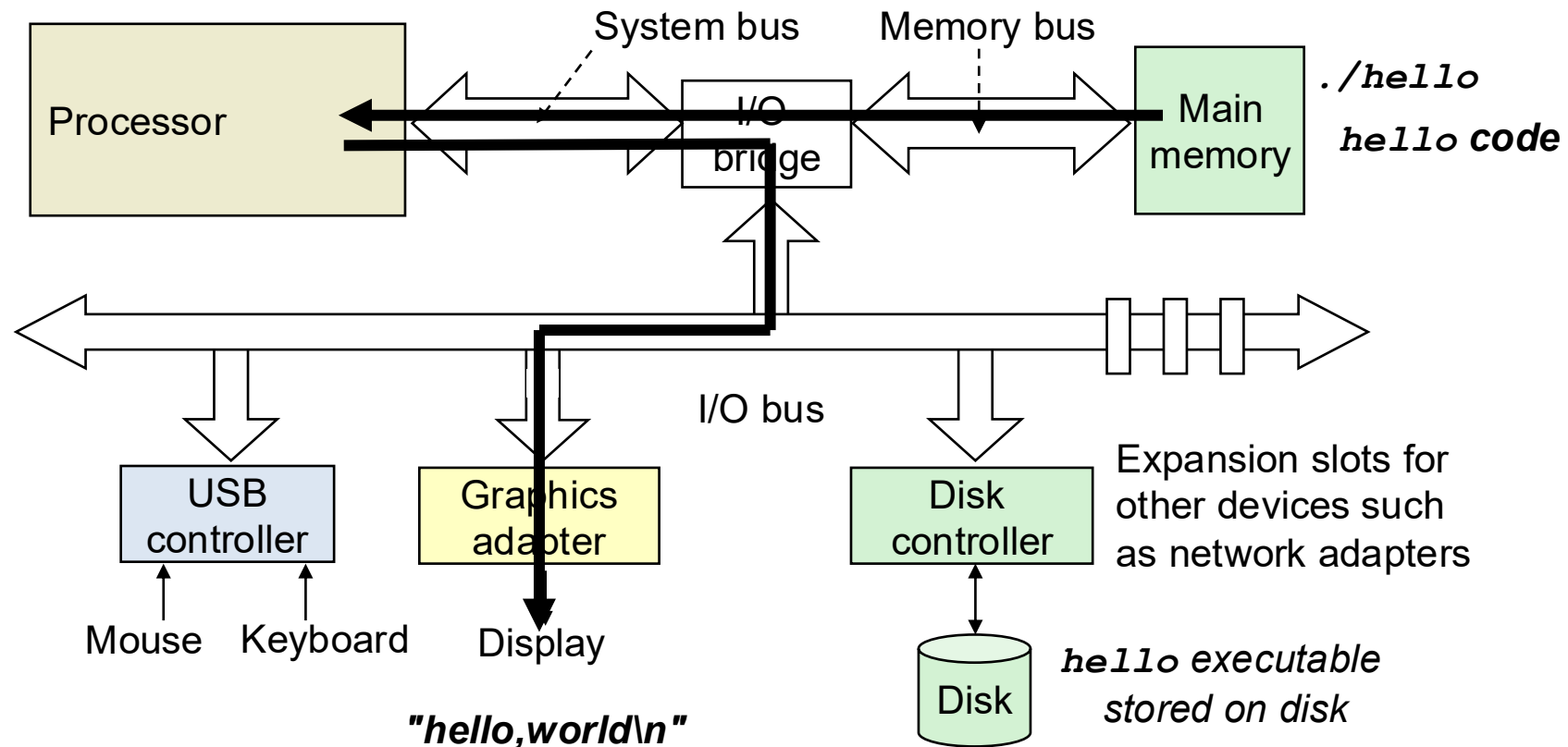
Running `hello`

Shell program loads the `hello` executable into main memory



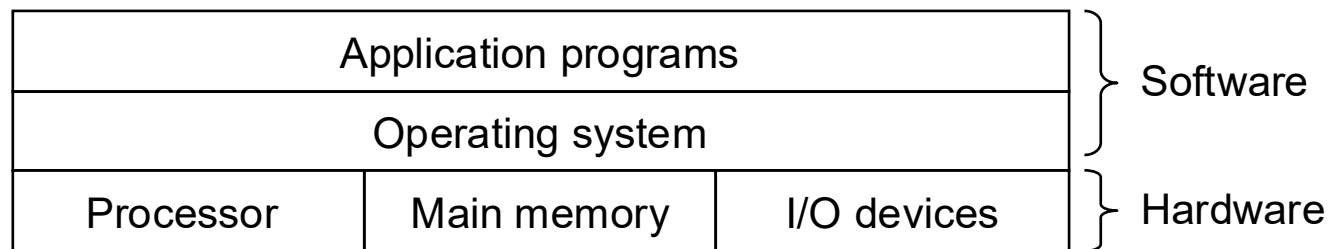
Running `hello`

The processor reads the `hello` code, executes instructions, and displays “hello...”



The Operating System (OS)

- Neither **hello** nor our shell interfaced with the hardware directly
 - All interactions were mediated by the **operating system**
- **Operating system:** a layer of software interposed between the application program and the hardware



- Primary goals
 - Protect resources from misuse by applications
 - Provide simple and uniform mechanisms for manipulating hardware devices
 - Manage sharing of resources between applications

Key idea: a computer system is more than just hardware

- A computer is a collection of intertwined hardware and software that must cooperate to achieve the end goal – running applications
 - **Hardware:** expensive, fast, immutable
 - **Software:** cheap (comparatively), slow, flexible
 - Different tradeoffs
 - So we'll use them for different roles!
- The rest of the course will expand on this

Question 1

- **What part of the hello example takes the longest to run on a computer?**

Question 1

- **What part of the hello example takes the longest to run on a computer?**
 - The user typing (seconds)
 - Maybe that's cheating and we should start after they hit enter

Question 1

- **What part of the hello example takes the longest to run on a computer?**
 - The user typing (seconds)
 - Maybe that's cheating and we should start after they hit enter
 - Almost certainly loading the program from disk (milliseconds)
 - Possibly sending text to graphics (microseconds to milliseconds)
 - Definitely not executing the code (nanoseconds to microseconds)
 - 1 second = 1000 milliseconds = 1000000 microseconds = 1000000000 nanoseconds

Amdahl's Law

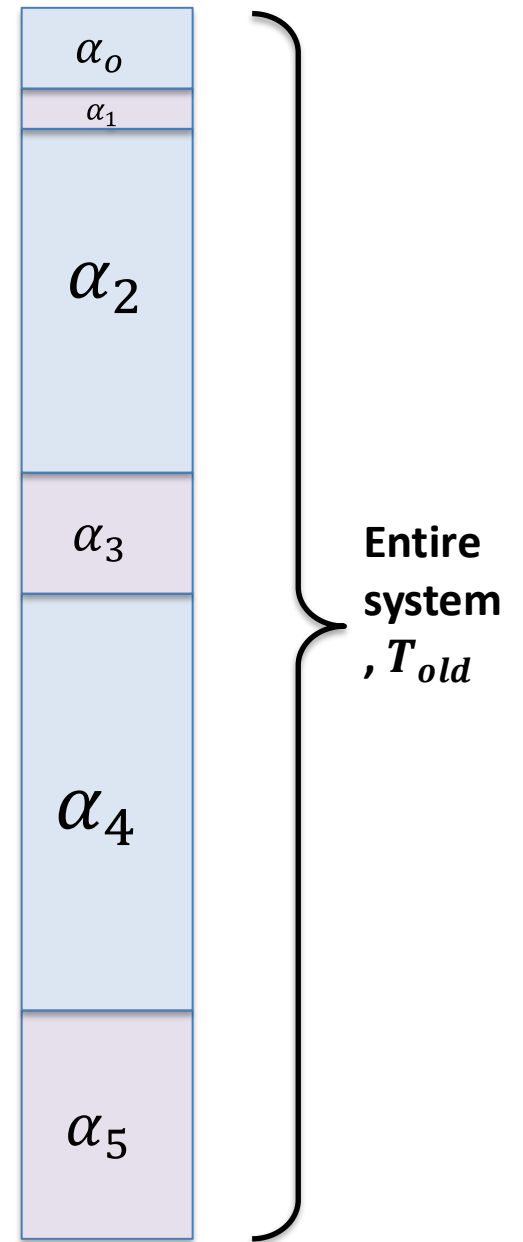
Describes the effectiveness of improving system performance by improving a part

- System requires some time, T_{old} , to execute an application
- A part of this system requires some fraction α of this total time
- We can speed up the part of the system that requires αT_{old} by k

$$T_{new} = (1 - \alpha)T_{old} + \frac{\alpha T_{old}}{k}$$

Speedup, S , is defined as:

$$S = \frac{T_{old}}{T_{new}} = \frac{1}{(1 - \alpha) + \frac{\alpha}{k}}$$



Question 2

- In the `hello` example, let's say loading the program from disk takes 60% of the total runtime. What is the speedup if we can load the program 3 times faster?

Question 2

- In the `hello` example, let's say loading the program from disk takes 60% of the total runtime. What is the speedup if we can load the program 3 times faster?
 - $S = \frac{T_{old}}{T_{new}} = \frac{1}{(1-\alpha) + \frac{\alpha}{k}}$
 - $S = \frac{T_{old}}{T_{new}} = \frac{1}{(1-0.6) + \frac{0.6}{3}} = \text{approx. 1.67x}$

Key takeaway: Even though we made a substantial improvement to a major part of the system, the net speedup was less significant!!

- Running a program
- **Representing numbers with binary**

Learning binary

- To understand how a computer really works we need to understand the data it operates on
- Computers hold data in memory as individual ones and zeros
 - These ones and zeros make up binary values
 - Bits are grouped together and combined with an interpretation that gives meaning to different bit patterns
- We're going to need to understand binary
 - With it, we will learn number representations, properties of arithmetic, and understand the ranges of values that can be represented in a computer system

Positional Numbering Systems

- The position of a *numeral* (e.g., digit) determines its contribution to the overall number
 - Makes arithmetic simpler (compared to, say, roman numerals)
 - Any number has one canonical representation
 - **Canonical form:** a standard way to represent a mathematical object in a unique way
- Example: base 10
 - $10456_{10} = 1*10^4 + 0*10^3 + 4*10^2 + 5*10^1 + 6*10^0$
 - Usually, we leave out the zeros:
 - $1*10^4 + 4*10^2 + 5*10^1 + 6*10^0 = 10456_{10}$

Other bases are also possible

- Base 60: used by the Babylonians
 - The source of 60 seconds in a minute, 60 minutes in an hour
 - And 360 degrees in a circle
- Base 20: used by the Maya and Gauls
 - Parts of this remain in French today
- Base 2: used by computers
 - Example: 10010010_2
 - Same idea as before: $1 \cdot 2^7 + 1 \cdot 2^4 + 1 \cdot 2^1 = 128_{10} + 16_{10} + 2_{10} = 146_{10}$
- Another word for **base** in a positional number system is **radix**
 - Digits are symbols representing all values less than the radix value
 - Each symbol represents unique quantity from zero through $r-1$, $[0, r-1]$

Base 2 Example

- Computer Scientists use base 2 a **LOT** (especially in computer systems)
- Let's convert 138_{10} to base 2
- We need to decompose 138_{10} into a sum of powers of 2
 - Start with the largest power of 2 that is smaller or equal to 138_{10}
 - Subtract it, then repeat the process

$$\begin{array}{rcl} 138_{10} - 128_{10} & = & 10_{10} \\ 10_{10} - 8_{10} & = & 2_{10} \\ 2_{10} - 2_{10} & = & 0_{10} \end{array}$$

$$138_{10} = \underline{1} \times 128 + 0 \times 64 + 0 \times 32 + 0 \times 16 + \underline{1} \times 8 + 0 \times 4 + \underline{1} \times 2 + 0 \times 1$$

$$138_{10} = \underline{1} \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + \underline{1} \times 2^3 + 0 \times 2^2 + \underline{1} \times 2^1 + 0 \times 2^0$$

$$138_{10} = 10001010_2$$

Binary practice

- Convert 101_2 to decimal

$$\circ = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

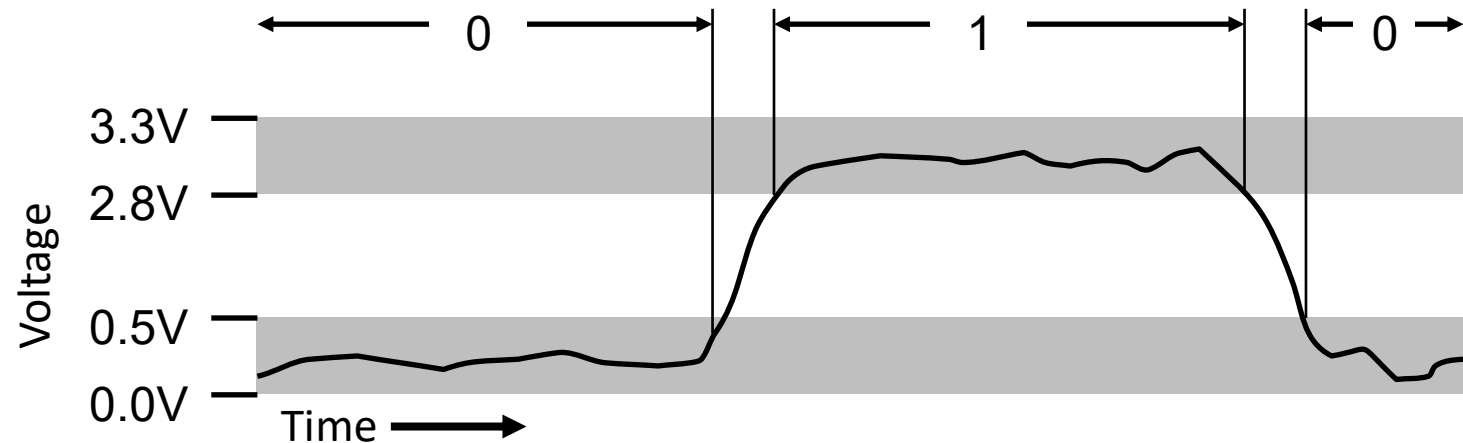
$$\circ = 4 + 0 + 1$$

$$\circ = 5_{10}$$

- Convert 4_{10} to binary: 100_2 (one less than 5)

Why computers use Base 2

- Simple electronic implementation
 - Easy to store with electronics that can have two states (e.g. bistable transistors)
 - Reliably transmitted on noisy and inaccurate wires



- Straightforward implementation of arithmetic functions
- (Pretty much) all computers use base 2
 - Although many communication methods use more states per transmitted symbol

Why don't computers use Base 10?

- Because implementing it electronically is a pain
 - Hard to store
 - ENIAC (first general-purpose electronic computer) used 10 vacuum tubes / digit
 - Hard to transmit
 - Need high precision to encode 10 signal levels on single wire
 - Messy to implement digital logic functions
 - Addition, multiplication, etc.
 - (See CE203 for details)

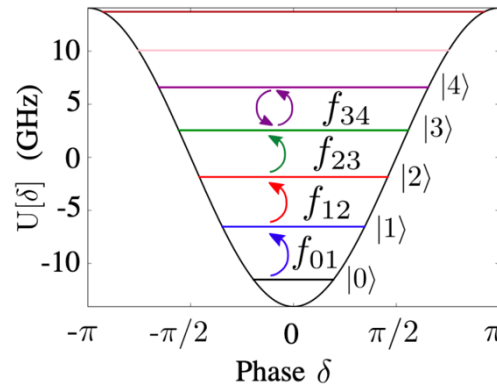


Aside: higher dimensional computation

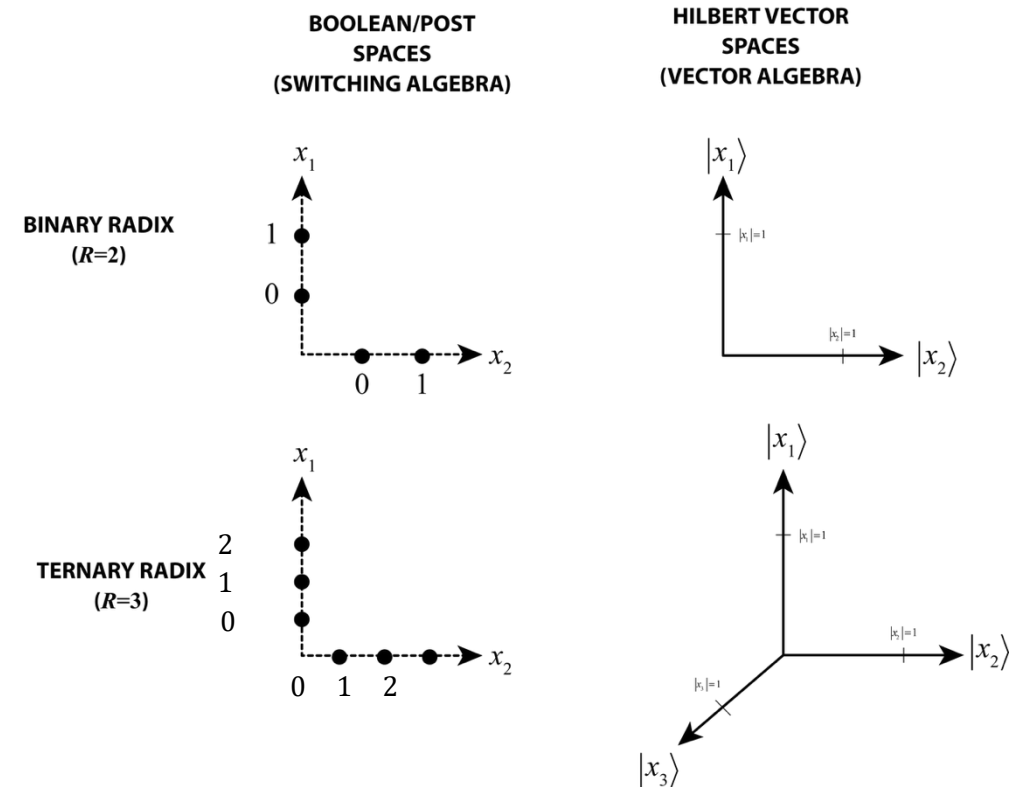
- More information per 'digit' as radix, r , increases
- Theoretically, less required hardware for M digits than N bits
- While it may not be practical for classical computing systems, higher-dimensional quantum systems are feasible!
 - Photonic: orbital angular momentum, time-energy, frequency, time-phase, location
 - Transmon: energy levels
 - Atoms and Ions: energy levels

$$M = \frac{N}{\log_2 r}$$

Data Compression



Simulated Energy Spectrum of a Transmon



Comparison of vector spaces for $r = 2, 3$

The binary number system

- Each digit in a binary number may represent one of two values, 0 or 1.
- N binary digits (N bits) can represent unsigned integers from 0 to $2^N - 1$.
 - 2^N total numbers for N bits (remember 0 is in this count!)
- Positional number system (like decimal) where:
 - Left values are larger
 - Right values are smaller
- Example:
 - 4 bits represents [0,15]
 - 8 bits represents values [0,255]
 - 16 bits represents values [0,65535]

Powers of two

n	2ⁿ	n	2ⁿ
0	1	8	256
1	2	9	512
2	4	10	1,024 (1K)
3	8	12	4,096 (4K)
4	16	20	1,048,576 (1M)
5	32	30	1,073,741,824 (1G)
6	64	40	1,099,511,627,776 (1 T)
7	128		

Binary number abbreviations (customary)

- 1 K (Kilo-) = $2^{10} = (1,024)_{10}$
 - NOT $10^3 = (1000)_{10}$ – Abbreviated 1k
- 1 M (Mega-) = $2^{20} = (1,048,576)_{10}$
- 1 G (Giga-) = $2^{30} = (1,073,741,824)_{10}$
- 1 T (Tera-) = $2^{40} = (1,099,511,627,776)_{10}$

Reminder: When multiplying, exponents are summed! (e.g. $2^8 * 2^2 = 2^{10}$)

More about binary prefixes

The traditional use of the units *kilobyte*, *megabyte*, and *gigabyte* by the computer industry sometimes causes confusion within the scientific community since their actual value differs from what is defined by the **International System of Units, SI**. To help clear confusion, the **International Electrotechnical Commission, IEC**, introduced a new set of binary prefixes.

Specific units of IEC 60027-2 A.2 and ISO/IEC 80000:13-2025

IEC prefix		Representations			
Name	Symbol	Base 2	Base 1024	Value	Base 10
kibi	Ki	2^{10}	1024^1	1024	$\approx 1.024 \times 10^3$
mebi	Mi	2^{20}	1024^2	1 048 576	$\approx 1.049 \times 10^6$
gibi	Gi	2^{30}	1024^3	1 073 741 824	$\approx 1.074 \times 10^9$
tebi	Ti	2^{40}	1024^4	1 099 511 627 776	$\approx 1.100 \times 10^{12}$
pebi	Pi	2^{50}	1024^5	1 125 899 906 842 624	$\approx 1.126 \times 10^{15}$
exbi	Ei	2^{60}	1024^6	1 152 921 504 606 846 976	$\approx 1.153 \times 10^{18}$
zebi	Zi	2^{70}	1024^7	1 180 591 620 717 411 303 424	$\approx 1.181 \times 10^{21}$
yobi	Yi	2^{80}	1024^8	1 208 925 819 614 629 174 706 176	$\approx 1.209 \times 10^{24}$
robi	Ri	2^{90}	1024^9	1 237 940 039 285 380 274 899 124 224	$\approx 1.238 \times 10^{27}$
quebi	Qi	2^{100}	1024^{10}	1 267 650 600 228 229 401 496 703 205 376	$\approx 1.268 \times 10^{30}$

In 2022, the [International Bureau of Weights and Measures](#) (BIPM) adopted the decimal prefixes *ronna* for 1000^9 and *quetta* for 1000^{10} .^{[6][7]} In 2025, the prefixes *robi* (Ri, 1024^9) and *quebi* (Qi, 1024^{10}) were adopted by the IEC.^[8]

Where confusion can happen...

- In computing, storage *typically* uses units for powers of two (customary prefix)
- Clock speeds, however reported with SI defined prefix: 1 GHz = 1,000,000,000 Hz **NOT** 1,073,741,824 Hz (= 2³⁰)

The relative difference between the values in the binary and decimal interpretations increases, when using the SI prefixes as the base, from 2.4% for kibi vs. kilo to nearly 27% for quebi vs. quetta.

Prefix		Binary ÷ Decimal		Decimal ÷ Binary	
kilo	kibi	1.024 (+2.4%)		0.9766 (−2.3%)	
mega	mebi	1.049 (+4.9%)		0.9537 (−4.6%)	
giga	gibi	1.074 (+7.4%)		0.9313 (−6.9%)	
tera	tebi	1.100 (+10.0%)		0.9095 (−9.1%)	
peta	pebi	1.126 (+12.6%)		0.8882 (−11.2%)	
exa	exbi	1.153 (+15.3%)		0.8674 (−13.3%)	
zetta	zebi	1.181 (+18.1%)		0.8470 (−15.3%)	
yotta	yobi	1.209 (+20.9%)		0.8272 (−17.3%)	
ronna	robi	1.238 (+23.8%)		0.8078 (−19.2%)	
quetta	quebi	1.268 (+26.8%)		0.7889 (−21.1%)	

Base 16: Hexadecimal

- Writing long sequences of 0s and 1s is tedious and error-prone
 - And takes up a lot of space on a page!
- Often preferred to use base 16 (also called *hexadecimal*)
- Base 2 = 2 symbols (0, 1)
Base 10 = 10 symbols (0-9)
Base 16, need 16 symbols
 - Use the digits 0 – 9
 - Use letters A – F once we run out of decimal digits

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Base 16: Hexadecimal

- $16 = 2^4$, so every group of 4 bits becomes a hexadecimal digit (or *hexit*)
 - If we have a number of bits not divisible by 4, add 0s on the left (always ok, just like base 10)

0010 1001 0111 1011 → 0x297B

“0x” prefix => it’s in hex

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Bytes

- A single bit doesn't hold much information
 - Only two possible values: 0 and 1
 - We typically work with larger groups of bits
- For convenience, we'll refer to groups of 8 bits as **bytes**
 - B is used to indicate a byte whereas b indicates bits
 - We usually work with multiples of 8 bits at a time
 - Conveniently, 8 bits = 2 hexits
 - 4 bits are sometimes referred to as a **nybble**
- Some examples
 - 1 byte: 0b01100111 = 0x67
 - 2 bytes: 11000100 00101111₂ = 0xC42F

"0b" prefix => it's in binary

"0x" prefix => it's in hex

Practice problem: Storage calculation

How many bits in 3GB ?

Practice problem: Storage calculation

How many bits in 3GB ?

$$3GB = 3 * 2^{30}B$$

$$3 * 2^{30}B * \frac{2^3 b}{1 B} = 3 * 2^{33} b$$

$$3 * 2^{33} b = 3 * 2^3 * 2^{30} b$$

$$3 * 8Gb$$

$$3 GB = 24 Gb = 25,769,803,776 \text{ bits}$$

Practice problem: Base conversion

- **Convert 0x42 to decimal**
- **Steps**
 - Convert 0x42 to binary:
 - Convert binary to decimal:

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice problem: Base conversion

- **Convert 0x42 to decimal**
- Steps
 - Convert 0x42 to binary:
 - 0x4 -> 0b0100 0x2 -> 0b0010
 - 0x42 -> 0b 0100 0010
 - Convert binary to decimal:

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice problem: Base conversion

- **Convert 0x42 to decimal**
- **Steps**
 - Convert 0x42 to binary:
 - 0x4 -> 0b0100 0x2 -> 0b0010
 - 0x42 -> 0b 0100 0010
 - Convert binary to decimal:
 - $1*2^6 + 1*2^1 = 64 + 2 = 66$

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Practice problem: Base conversion

- **Convert 0x42 to decimal**
- Alternate method:
 - 0x42
 - $= 4 \times 16^1 + 2 \times 16^0$
 - $= 64 + 2$
 - $= 66$
- But you're honestly better off converting hex to binary first
 - It's good practice!

The range for an encoded hex value

When considering four bits (positive integers only), what are the maximum and minimum values in decimal?

- Minimum 0 (0b0000)
- Maximum 15 (0b1111)

When considering two hex digits (positive integers only), what are the maximum and minimum values in decimal?

- Minimum 0 (0x00)
- Maximum 255 (0xFF)

N hex digits (remember, that's $4 \cdot N$ bits) can represent unsigned integers from 0 to $16^N - 1$ (includes zero)

- $16^2 - 1 = 255$

Important idea: bits can be used to represent anything

- Depending on the context, the bits **11000011** could mean
 - The number 195
 - The number -61
 - The number -1.1875
 - The value True
 - The character '┘'
 - The **ret** x86 instruction
- You have to know the **context** to make sense of any bits you have!
 - People and software they write determine what the bits actually mean

More practice: Base conversion

Remember to start counting bit positions from 0!!

- Translate 0b10111 to decimal (0b means binary)
 - $1*16 + 0*8 + 1*4 + 1*2 + 1*1$
 - $16 + 4 + 2 + 1$
 - 23

Remember to start counting powers of two from 1!!

- Translate decimal 40 to binary
 - $32 + 8$
 - $1*32 + 0*16 + 1*8 + 0*4 + 0*2 + 0*1$
 - 0b101000

Decimal to binary: General method

Continuously divide by 2, keeping track of remainders

Example: convert 26_{10} to binary.

$$26 \div 2 = 13, R = 0$$

$$13 \div 2 = 6, R = 1$$

$$6 \div 2 = 3, R = 0$$

$$3 \div 2 = 1, R = 1$$

$$1 \div 2 = 0, R = 1$$

Digit position of new number moves away from decimal point as number of remainders increases!! (Start with least significant digit, end with most significant)

Result: 0b11010

$$\text{Check: } 1 * 2^4 + 1 * 2^3 + 1 * 2^1 = 26_{10}$$

Bit widths

- Since information must be physically encoded, we cannot encode arbitrary bit lengths
- Example: convert 0x34 into a 6-bit binary number
 - 0x34 -> 0b0011 0100 -> 0b110100
- Can you convert 0x74 into a 6-bit value?
 - No! 0x74 = 0b0111 0100
 - While a conversion could happen, we'd have an overflow!
- It is easy to make a number larger – just pad with zeros
 - Q: Can you convert 0x34 to a 16-bit value?
 - A: 0b 0000 0000 0011 0100